AI Bootcamp — Week 1: Foundations

Tuesday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 1, the candidate can:

- Explain the Prompt/Harness/Model framework and why the harness, not the model, controls what a skill can do
- Apply the EDD (Evaluation-Driven Development) discipline: write Evaluation Criteria (EC) and manual test cases before building any skill
- Explain and apply the RTCC prompt structure (Role = WHO, Task = WHAT, Context = WHY, Constraints = HOW) to a real skill
- Build four sequential content creation skills as durable, committed `.skill.md` files
- Direct an AI to write a Python evaluation script (deterministic checks plus an LLM judge) and interpret its results
- Verify that every instruction in a skill is load-bearing using manual tests and evaluation runs
- Produce a working end-to-end content pipeline: Trend Identification → Ideation → Development → Graphics

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning, 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon, 3 hours)

© 2026 Improving

TUESDAY (9:00 AM – 5:00 PM): TREND IDENTIFICATION SKILL

9:00 – 10:00 AM — THE PIPELINE AND SKILL 1 BRIEF

Overview of the full week pipeline. Each skill produces a durable Markdown file with a defined schema. **Skill 1: Trend Identification:**

- **Input:** A topic area or realm (e.g., "AI in software development", "sustainable fashion", "Formula 1")
- **Output file:** `trends.md` : a structured list of current trending topics and hashtags within that realm
- **Required output elements:** Topic name, why it's trending (one sentence, evidence-based or model-knowledge-based), 3–5 hashtags per topic in `#format` , platform relevance notes (LinkedIn vs. X/Twitter vs. Instagram differ)

Skill file anatomy. A skill file is a Markdown file with YAML frontmatter:

```
---
name: trend-identification
description: Identify current trending topics and hashtags within a given realm
disable-model-invocation: true
---
```

The body below the frontmatter is your RTCC prompt. `disable-model-invocation: true` means the harness runs this skill only when you invoke it explicitly (via slash command). The `description` tells the harness, and your future self, what the skill is for.

10:00 – 11:30 AM — EDD FIRST: WRITE THE TEST FILE

Before opening your IDE, write the manual test file for the Trend Identification skill. Do this on paper or in a plain text file, not in the IDE, not with AI assistance.

For at least 3 different topic areas, define:

- The exact input you will provide
- The EC the `trends.md` output must meet to pass (structure, content, length, format, freshness signals)
- At least one explicit failure criterion: something the output must NOT contain or do

Commit this test file before writing a single line of the skill. The test file is your Red state.

11:30 AM – 1:00 PM — BUILD THE TREND IDENTIFICATION SKILL

Write the Trend Identification skill as an RTCC-structured `.skill.md` file with the frontmatter above. Apply the load-bearing rule: every sentence you add should correspond to at least one EC. If you're adding language that no EC would catch, either add the EC or don't add the language.

Commit the skill file.

© 2026 Improving

1:00 – 2:00 PM — LUNCH

2:00 – 3:30 PM — RUN AND ITERATE

Run the skill against your test inputs. Grade each output against your EC. Record results in the test file (pass/fail per criterion per input). Identify the weakest criterion. Form a specific hypothesis: *"The output is failing criterion X because the prompt does not specify Y."* Make one targeted change. Re-run. Repeat at least twice.

Rule: One change per iteration. Document the hypothesis before making the change.

3:30 – 5:00 PM — AI-WRITTEN PYTHON EVALUATION SCRIPTS

Manual grading taught you what your EC actually mean. Now you need a measurement tool you can run on every iteration for the rest of this program: a **Python evaluation script**.

The key discipline: **the AI writes the code; you direct and review.** There is no Python prerequisite. Your job is to bring the EC (which you wrote by hand, before the skill existed) and to read what the AI produces closely enough to explain every check it contains. The models carry the coding load.

Two kinds of checks, in this order:

1. **Deterministic checks first.** Schema, counts, format, structure. These run without any model call: fast, free, definitive. Examples for this skill: does `trends.md` contain 5–8 numbered topics? Does each topic have 3–5 hashtags in `#format`?
2. **LLM judge second.** For EC that genuinely require qualitative judgment, the script sends the skill output plus the EC text to a model and gets back a structured JSON judgment: pass/fail, score, and reasoning per criterion. The rubric text should be the same EC you wrote in your manual test file. If your manual EC were vague, the judge's verdicts will be vague too. This is the feedback loop.

> *If you find yourself using an LLM judge for something that could be a regex, your EC needs refinement.*

Exercise:

- Direct the AI to write an evaluation script for your Trend Identification skill: at least 2 deterministic checks and 1 LLM judge check, driven by your manual test file.
- Read the script. Be able to explain what each check does and why it is deterministic or judged. (The certification gate requires you to *demonstrate* this, not describe it.)
- Run the script against your current skill's output. Record the baseline result. This is your Red state for tomorrow.

Commit the evaluation script alongside the skill it tests.

© 2026 Improving